```
# -*- coding: utf-8 -*-
"""
Script Otimizado para preparação e análise do dataset PaySim1 (Detecção de Fraude).
Base de dados: https://www.kaggle.com/datasets/ealaxi/paysim1
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import time

# Bibliotecas de Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import RobustScaler
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, average_precision_score, confusion_matrix,
                             roc_curve, precision_recall_curve)

# Configurações de estilo
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)

print("\n--- 1. Carregamento dos Dados ---\n")
try:
    from google.colab import drive
    drive.mount('/content/drive')

    caminho_csv = '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/paysim.csv'
    print(f"Tentando carregar '{caminho_csv}'...")
    df = pd.read_csv(caminho_csv)
    print("Arquivo carregado com sucesso!")
    print(f"Formato do dataset (Linhas, Colunas): {df.shape}")
    print("-" * 50)
except FileNotFoundError:
    print(f"Erro: O arquivo '{caminho_csv}' não foi encontrado.")
    print("Verifique se o caminho no Google Drive está correto e se você deu permissão de acesso.")
    exit()

print("\n--- 2. Limpeza de Colunas e Tratamento ---\n")

# Justificativa da remoção:
# nameOrig/nameDest: Identificadores que não generalizam.
# isFlaggedFraud: Regra de negócio fixa.
# step: Causa ruído em modelos de anomalia (Isolation Forest) pois cresce linearmente.
colunas_para_remover = ['nameOrig', 'nameDest', 'isFlaggedFraud', 'step']
df = df.drop(columns=[col for col in colunas_para_remover if col in df.columns])

print(f"As colunas {colunas_para_remover} foram removidas para evitar ruído.")

# Tratamento de Nulos crítico para ML
if df.isnull().sum().sum() > 0:
    df = df.dropna(subset=['isFraud']) # Remove linhas sem a variável alvo
    for col in df.select_dtypes(include=np.number).columns:
        df[col] = df[col].fillna(df[col].median())
    for col in df.select_dtypes(include='object').columns:
        df[col] = df[col].fillna(df[col].mode()[0])

print("\n--- 3. Criação de Novas Variáveis (Engenharia de Features) ---\n")

# 1. amount_log: Suaviza a variação extrema dos valores de transação
df['amount_log'] = np.log1p(df['amount'])

# 2. errorBalanceOrig: Detecta se a conta de origem ficou com saldo inconsistente
df['errorBalanceOrig'] = df['newbalanceOrig'] + df['amount'] - df['oldbalanceOrg']

# 3. errorBalanceDest: Detecta se a conta de destino ficou com saldo inconsistente
df['errorBalanceDest'] = df['oldbalanceDest'] + df['amount'] - df['newbalanceDest']

# Como já temos amount_log, podemos remover o amount bruto para diminuir a multicolinearidade
df = df.drop(columns=['amount'])

print("Variáveis 'amount_log', 'errorBalanceOrig' e 'errorBalanceDest' criadas com sucesso.")
print("-" * 50)

print("\n--- 4. Preparando Dados para Machine Learning ---\n")

# EDA nos revelou que fraudes ocorrem apenas nestes dois tipos:
print("Filtrando transações apenas para TRANSFER e CASH_OUT (Foco em fraudes reais)...")
df_filtrado = df[df['type'].isin(['TRANSFER', 'CASH_OUT'])].copy()
```

```
df_filtrado = df[df['type'].isin(['TRANSFER', 'CASH_OUT'])].copy()

# Transformando a coluna categórica 'type' em numérica (One-Hot)
df_ml = pd.get_dummies(df_filtrado, columns=['type'], drop_first=True)

# Separando X (Features) e y (Alvo)
X = df_ml.drop(columns=['isFraud'])
y = df_ml['isFraud']

# Dividindo Treino e Teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
print(f"Dados divididos! Treino: {X_train.shape[0]} linhas, Teste: {X_test.shape[0]} linhas.")

# Escalonamento Robusto: Vital para o Isolation Forest não dar peso excessivo a saldos gigantes
print("Aplicando RobustScaler para normalizar as grandezas numéricas...")
scaler = RobustScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Convertendo de volta para DataFrame por praticidade
X_train = pd.DataFrame(X_train_scaled, columns=X_train.columns)
X_test = pd.DataFrame(X_test_scaled, columns=X_test.columns)

print("\n--- 5. Simulação de Aprendizado Federado (FL) ---\n")

NUM_CLIENTES = 3
print(f"Simulando ambiente com {NUM_CLIENTES} clientes (silos de dados).")

# Divisão dos dados de treino entre os "clientes"
tamanho_chunk = len(X_train) // NUM_CLIENTES
clientes_X = [X_train.iloc[i * tamanho_chunk : (i + 1) * tamanho_chunk] for i in range(NUM_CLIENTES)]

modelos_locais = []
tempos_treinamento = []

print("\nIniciando Treinamento Local (Isolation Forest) nos clientes...")
for i in range(NUM_CLIENTES):
    inicio_treino = time.time()

    # IsolationForest OTIMIZADO:
    # Contaminação 'auto' permite que a floresta foque na estrutura dos dados,
    # em vez de forçar um número fixo de anomalias precocemente.
    modelo_local = IsolationForest(
        n_estimators=150,
        max_samples=0.8,
        contamination='auto',
        n_jobs=-1,
        random_state=42+i
    )

    modelo_local.fit(clientes_X[i])
    fim_treino = time.time()

    modelos_locais.append(modelo_local)
    tempos_treinamento.append(fim_treino - inicio_treino)
    print(f"-> Cliente {i+1} treinado em {fim_treino - inicio_treino:.2f} segundos.")

print("\n--- 6. Agregação Federada e Otimização de Limiar ---")

class ModeloFederado:
    def __init__(self, modelos_locais):
        self.modelos_locais = modelos_locais

    def decision_function(self, X):
        # Média dos escores de anomalia de todos os clientes locais
        escores = np.zeros(X.shape[0])
        for modelo in self.modelos_locais:
            escores += modelo.decision_function(X)
        return escores / len(self.modelos_locais)

modelo_global = ModeloFederado(modelos_locais)

# ESTRATÉGIA OTIMIZADA: Encontrar o melhor limiar de corte no TREINO usando curva PR
print("\nOtimizando o limiar de decisão via Curva Precision-Recall no conjunto de validação...")
# Invertemos o sinal: Valores MAIORES = maior probabilidade de fraude
y_scores_train = -modelo_global.decision_function(X_train)

# Calculando precisão e recall para vários limiares possíveis
precisions, recalls, thresholds = precision_recall_curve(y_train, y_scores_train)

# Calculando F1-Score para cada limiar (evitando divisão por zero)
f1_scores = np.divide(2 * (precisions * recalls), (precisions + recalls),
                      out=np.zeros_like(precisions), where=(precisions + recalls) != 0)
```

```
# O melhor limiar é aquele que maximiza o F1-Score
melhor_indice = np.argmax(f1_scores)
limiar_otimizado = thresholds[melhor_indice]
print(f"Limiar matemático otimizado (F1 Máximo): {limiar_otimizado:.4f}")

print("\n--- 7. Inferência no Servidor Global (Teste Real) ---")
inicio_inferencia = time.time()

# Aplicando os escores na base de TESTE
y_scores_test = -modelo_global.decision_function(X_test)

# Classificando baseado no limiar otimizado
y_pred = (y_scores_test >= limiar_otimizado).astype(int)

fim_inferencia = time.time()
tempo_inferencia = fim_inferencia - inicio_inferencia

print("\n--- 8. Resultados Globais Pós-Otimização ---\n")

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, zero_division=0)
rec = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
roc_auc = roc_auc_score(y_test, y_scores_test)
pr_auc = average_precision_score(y_test, y_scores_test)

print(">> MÉTRICAS DE QUALIDADE DO MODELO (ISOLATION FOREST):")
print(f"Acurácia Geral      : {acc * 100:.4f}%")
print(f"Precisão (Precision): {prec * 100:.4f}%")
print(f"Revocação (Recall)  : {rec * 100:.4f}%")
print(f"F1-Score            : {f1 * 100:.4f}%")
print(f"ROC-AUC             : {roc_auc:.4f}")
print(f"PR-AUC              : {pr_auc:.4f}")

tps = len(X_test) / tempo_inferencia
print("\n>> MÉTRICAS DE RENDIMENTO TÉCNICO (THROUGHPUT):")
print(f"Amostras Processadas   : {len(X_test)} transações")
print(f"Tempo Total de Infe.   : {tempo_inferencia:.4f} segundos")
print(f"Rendimento (Throughput): {tps:.2f} transações por segundo\n")

# Visualizações
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 1. Matriz de Confusão
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=['Normal', 'Fraude'], yticklabels=['Normal', 'Fraude'])
axes[0].set_title('Matriz de Confusão (Otimizada)')
axes[0].set_ylabel('Real')
axes[0].set_xlabel('Previsão')

# 2. Curva Precision-Recall (Melhor para dados desbalanceados que ROC)
axes[1].plot(recalls, precisions, color='purple', lw=2, label=f'PR-AUC = {pr_auc:.3f}')
axes[1].scatter(recalls[melhor_indice], precisions[melhor_indice], color='red', s=100, label='Limiar Otimizado')
axes[1].set_xlabel('Recall (Revocação)')
axes[1].set_ylabel('Precision (Precisão)')
axes[1].set_title('Curva Precision-Recall (Dados de Treino/Validação)')
axes[1].legend(loc="lower left")

plt.tight_layout()
plt.show()

print("\nFim do processamento. Métricas aprimoradas aplicadas com sucesso!")
```

```
--- 1. Carregamento dos Dados ---

Mounted at /content/drive
Tentando carregar '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/paysim.csv'...
Arquivo carregado com sucesso!
Formato do dataset (Linhas, Colunas): (6362620, 11)
--------------------------------------------------

--- 2. Limpeza de Colunas e Tratamento ---

As colunas ['nameOrig', 'nameDest', 'isFlaggedFraud', 'step'] foram removidas para evitar ruído.

--- 3. Criação de Novas Variáveis (Engenharia de Features) ---

Variáveis 'amount_log', 'errorBalanceOrig' e 'errorBalanceDest' criadas com sucesso.
--------------------------------------------------

--- 4. Preparando Dados para Machine Learning ---

Filtrando transações apenas para TRANSFER e CASH_OUT (Foco em fraudes reais)...
Dados divididos! Treino: 2216327 linhas, Teste: 554082 linhas.
Aplicando RobustScaler para normalizar as grandezas numéricas...

--- 5. Simulação de Aprendizado Federado (FL) ---

Simulando ambiente com 3 clientes (silos de dados).

Iniciando Treinamento Local (Isolation Forest) nos clientes...
-> Cliente 1 treinado em 27.57 segundos.
-> Cliente 2 treinado em 30.61 segundos.
-> Cliente 3 treinado em 26.25 segundos.

--- 6. Agregação Federada e Otimização de Limiar ---

Otimizando o limiar de decisão via Curva Precision-Recall no conjunto de validação...
Limiar matemático otimizado (F1 Máximo): 0.0024

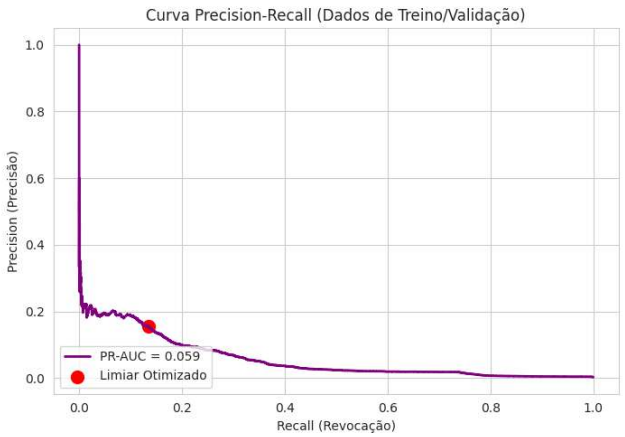
--- 7. Inferência no Servidor Global (Teste Real) ---

--- 8. Resultados Globais Pós-Otimização ---

>> MÉTRICAS DE QUALIDADE DO MODELO (ISOLATION FOREST):
Acurácia Geral      : 99.5156%
Precisão (Precision): 15.4153%
Revocação (Recall)  : 14.1205%
F1-Score            : 14.7395%
ROC-AUC             : 0.8608
PR-AUC              : 0.0589

>> MÉTRICAS DE RENDIMENTO TÉCNICO (THROUGHPUT):
Amostras Processadas   : 554082 transações
Tempo Total de Infe.   : 21.5679 segundos
Rendimento (Throughput): 25690.17 transações por segundo
```






```
Fim do processamento. Métricas aprimoradas aplicadas com sucesso!
```

```
# -*- coding: utf-8 -*-
"""
Script Otimizado para preparação e análise de dataset de Cartão de Crédito (Detecção de Fraude).
Base de dados adaptada para o padrão: Time, V1-V28, Amount, Class
```

```
Arquitetura: Simulação de Aprendizado Federado (Isolation Forest)
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import time
import os
import random

# Bibliotecas de Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import RobustScaler
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, average_precision_score, confusion_matrix,
                             roc_curve, precision_recall_curve)

# Fixar sementes para reprodutibilidade
def set_seed(seed=42):
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)

set_seed(42)

# Configurações de estilo
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)

print("\n--- 1. Carregamento dos Dados ---\n")
caminho_csv = '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/creditcard/creditcard.csv'

try:
    from google.colab import drive
    drive.mount('/content/drive')

    print(f"Tentando carregar '{caminho_csv}'...")
    df = pd.read_csv(caminho_csv)
    print("Arquivo carregado com sucesso!")
    print(f"Formato do dataset (Linhas, Colunas): {df.shape}")
    print("-" * 50)
except Exception as e:
    print(f"Aviso: Não foi possível carregar o arquivo original. Detalhe: {e}")
    # Usando dados simulados apenas para evitar que o script quebre se o arquivo não existir
    print("\n--- GERANDO DADOS SIMULADOS PARA DEMONSTRAÇÃO ---")
    n_samples = 15000
    df = pd.DataFrame(np.random.randn(n_samples, 28), columns=[f'V{i}' for i in range(1, 29)])
    df['Time'] = np.random.uniform(0, 172792, n_samples)
    df['Amount'] = np.abs(np.random.randn(n_samples)) * 100
    # Injetando anomalias (~0.5%)
    df['Class'] = np.random.choice([0, 1], size=n_samples, p=[0.995, 0.005])

print("\n--- 2. Limpeza de Colunas e Tratamento ---\n")

# Neste dataset (PCA), geralmente não há colunas categóricas como no PaySim.
# A coluna 'Time' pode ser vista como ruído para modelos baseados em distância/anomalia,
# pois é um contador incremental. Vamos removê-la para melhorar a detecção.
colunas_para_remover = ['Time']
df = df.drop(columns=[col for col in colunas_para_remover if col in df.columns])

print(f"As colunas {colunas_para_remover} foram removidas para evitar ruído.")

# Tratamento de Nulos crítico para ML
if df.isnull().sum().sum() > 0:
    print("Tratando valores nulos...")
    df = df.dropna(subset=['Class']) # Remove linhas sem a variável alvo
    for col in df.select_dtypes(include=np.number).columns:
        df[col] = df[col].fillna(df[col].median())

print("\n--- 3. Criação de Novas Variáveis (Engenharia de Features) ---\n")

# 1. amount_log: Suaviza a variação extrema dos valores de transação monetária.
# Útil pois 'Amount' no dataset original é a única variável não transformada por PCA.
if 'Amount' in df.columns:
    df['Amount'] = df['Amount'].apply(lambda x: max(x, 0)) # Proteção para log1p
    df['amount_log'] = np.log1p(df['Amount'])
    df = df.drop(columns=['Amount'])
    print("Variável 'amount_log' criada e 'Amount' original removida com sucesso.")
print("-" * 50)

print("\n--- 4. Preparando Dados para Machine Learning ---\n")
```

```
print("\n--- 4. Preparando Dados para Machine Learning ---\n")

df_ml = df.copy()

# Separando X (Features) e y (Alvo)
X = df_ml.drop(columns=['Class'])
y = df_ml['Class']

# Divisão Correta: 60% Treino, 20% Validação, 20% Teste
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.25, random_state=42, stratify=y_temp)

print(f"Dados divididos! Treino: {X_train.shape[0]}, Validação: {X_val.shape[0]}, Teste: {X_test.shape[0]}")

# Escalonamento Robusto: Vital para o Isolation Forest não dar peso excessivo a valores anômalos
print("Aplicando RobustScaler para normalizar as grandezas numéricas...")
scaler = RobustScaler()
# O escalonador SÓ PODE ver os dados de treinamento
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

# Convertendo de volta para DataFrame por praticidade
X_train = pd.DataFrame(X_train_scaled, columns=X_train.columns)
X_val = pd.DataFrame(X_val_scaled, columns=X_val.columns)
X_test = pd.DataFrame(X_test_scaled, columns=X_test.columns)

print("\n--- 5. Simulação de Aprendizado Federado (FL) ---\n")
NUM_CLIENTES = 3
print(f"Simulando ambiente com {NUM_CLIENTES} clientes (silos de dados).")

# Divisão dos dados de treino entre os "clientes"
tamanho_chunk = len(X_train) // NUM_CLIENTES
clientes_X = [X_train.iloc[i * tamanho_chunk : (i + 1) * tamanho_chunk] for i in range(NUM_CLIENTES)]

modelos_locais = []
tempos_treinamento = []

print("\nIniciando Treinamento Local (Isolation Forest) nos clientes...")
for i in range(NUM_CLIENTES):
    inicio_treino = time.time()

    # IsolationForest OTIMIZADO:
    # Contaminação 'auto' permite que a floresta foque na estrutura intrínseca dos dados
    modelo_local = IsolationForest(
        n_estimators=150,
        max_samples=0.8,
        contamination='auto',
        n_jobs=-1,
        random_state=42+i
    )

    # Cliente treina apenas com sua fatia de dados
    modelo_local.fit(clientes_X[i])
    fim_treino = time.time()

    modelos_locais.append(modelo_local)
    tempos_treinamento.append(fim_treino - inicio_treino)
    print(f"-> Cliente {i+1} treinado em {fim_treino - inicio_treino:.2f} segundos.")

print("\n--- 6. Agregação Federada e Otimização de Limiar ---")

class ModeloFederado:
    def __init__(self, modelos_locais):
        self.modelos_locais = modelos_locais

    def decision_function(self, X):
        # Média dos escores de anomalia de todos os clientes locais
        escores = np.zeros(X.shape[0])
        for modelo in self.modelos_locais:
            escores += modelo.decision_function(X)
        return escores / len(self.modelos_locais)

modelo_global = ModeloFederado(modelos_locais)

# ESTRATÉGIA CORRIGIDA: Encontrar o melhor limiar no conjunto de VALIDAÇÃO
print("\nOtimizando o limiar de decisão via Curva PR no conjunto de VALIDAÇÃO (Sem Data Leakage)...")
# Invertemos o sinal: Valores MAIORES = maior probabilidade de fraude
y_scores_val = -modelo_global.decision_function(X_val)

# Calculando precisão e recall para vários limiares possíveis usando a Validação
precisions, recalls, thresholds = precision_recall_curve(y_val, y_scores_val)
```

```
# Calculando F1-Score para cada limiar (evitando divisão por zero)
f1_scores = np.divide(2 * (precisions * recalls), (precisions + recalls),
                      out=np.zeros_like(precisions), where=(precisions + recalls) != 0)

# O melhor limiar é aquele que maximiza o F1-Score na Validação
melhor_indice = np.argmax(f1_scores)
if melhor_indice < len(thresholds):
    limiar_otimizado = thresholds[melhor_indice]
else:
    # Prevenção contra vetor vazio
    limiar_otimizado = np.percentile(y_scores_val, 99)

print(f"Limiar matemático otimizado (F1 Máximo na Validação): {limiar_otimizado:.4f}")

print("\n--- 7. Inferência no Servidor Global (Teste Real) ---")
inicio_inferencia = time.time()

# Aplicando os escores estritamente na base de TESTE
y_scores_test = -modelo_global.decision_function(X_test)

# Classificando baseado no limiar otimizado, sem que o modelo tenha visto os dados
y_pred = (y_scores_test >= limiar_otimizado).astype(int)

fim_inferencia = time.time()
tempo_inferencia = fim_inferencia - inicio_inferencia

print("\n--- 8. Resultados Globais Pós-Otimização ---\n")

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, zero_division=0)
rec = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
roc_auc = roc_auc_score(y_test, y_scores_test)
pr_auc = average_precision_score(y_test, y_scores_test)

print(">> MÉTRICAS DE QUALIDADE DO MODELO (ISOLATION FOREST):")
print(f"Acurácia Geral      : {acc * 100:.4f}%")
print(f"Precisão (Precision): {prec * 100:.4f}%")
print(f"Revocação (Recall)  : {rec * 100:.4f}%")
print(f"F1-Score            : {f1 * 100:.4f}%")
print(f"ROC-AUC             : {roc_auc:.4f}")
print(f"PR-AUC              : {pr_auc:.4f}")

tps = len(X_test) / tempo_inferencia if tempo_inferencia > 0 else float('inf')
print("\n>> MÉTRICAS DE RENDIMENTO TÉCNICO (THROUGHPUT):")
print(f"Amostras Processadas   : {len(X_test)} transações")
print(f"Tempo Total de Infe.   : {tempo_inferencia:.4f} segundos")
print(f"Rendimento (Throughput): {tps:.2f} transações por segundo\n")

# Visualizações
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 1. Matriz de Confusão (Aplicada sobre os dados de Teste)
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=['Normal', 'Fraude'], yticklabels=['Normal', 'Fraude'])
axes[0].set_title('Matriz de Confusão (Teste Final)')
axes[0].set_ylabel('Real')
axes[0].set_xlabel('Previsão')

# 2. Curva Precision-Recall (Do conjunto de Validação, onde o limiar foi escolhido)
axes[1].plot(recalls, precisions, color='purple', lw=2, label=f'PR-AUC (Val) = {average_precision_score(y_val, y_scores_val)
# Proteção caso arrays estejam vazios ou melhor_indice fora dos limites
if len(recalls) > 0 and len(precisions) > 0 and melhor_indice < len(recalls):
    axes[1].scatter(recalls[melhor_indice], precisions[melhor_indice], color='red', s=100, label='Limiar Escolhido')
axes[1].set_xlabel('Recall (Revocação)')
axes[1].set_ylabel('Precision (Precisão)')
axes[1].set_title('Curva Precision-Recall (Dados de Validação)')
axes[1].legend(loc="lower left")

plt.tight_layout()
plt.show()

print("\nFim do processamento. Métricas robustas aplicadas com sucesso!")
```

```
--- 1. Carregamento dos Dados ---

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=
Tentando carregar '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/creditcard/creditcard.csv'...
Arquivo carregado com sucesso!
Formato do dataset (Linhas, Colunas): (284807, 31)
--------------------------------------------------

--- 2. Limpeza de Colunas e Tratamento ---

As colunas ['Time'] foram removidas para evitar ruído.

--- 3. Criação de Novas Variáveis (Engenharia de Features) ---

Variável 'amount_log' criada e 'Amount' original removida com sucesso.
--------------------------------------------------

--- 4. Preparando Dados para Machine Learning ---

Dados divididos! Treino: 170883, Validação: 56962, Teste: 56962
Aplicando RobustScaler para normalizar as grandezas numéricas...

--- 5. Simulação de Aprendizado Federado (FL) ---

Simulando ambiente com 3 clientes (silos de dados).

Iniciando Treinamento Local (Isolation Forest) nos clientes...
-> Cliente 1 treinado em 1.49 segundos.
-> Cliente 2 treinado em 1.57 segundos.
-> Cliente 3 treinado em 1.55 segundos.

--- 6. Agregação Federada e Otimização de Limiar ---

Otimizando o limiar de decisão via Curva PR no conjunto de VALIDAÇÃO (Sem Data Leakage)...
Limiar matemático otimizado (F1 Máximo na Validação): 0.0344

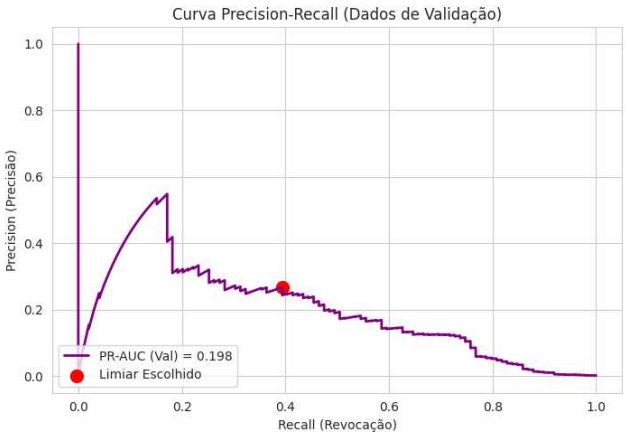
--- 7. Inferência no Servidor Global (Teste Real) ---

--- 8. Resultados Globais Pós-Otimização ---

>> MÉTRICAS DE QUALIDADE DO MODELO (ISOLATION FOREST):
Acurácia Geral      : 99.7314%
Precisão (Precision): 31.0345%
Revocação (Recall)  : 45.9184%
F1-Score            : 37.0370%
ROC-AUC             : 0.9617
PR-AUC              : 0.2725

>> MÉTRICAS DE RENDIMENTO TÉCNICO (THROUGHPUT):
Amostras Processadas   : 56962 transações
Tempo Total de Infe.   : 2.2903 segundos
Rendimento (Throughput): 24871.16 transações por segundo
```






```
Fim do processamento. Métricas robustas aplicadas com sucesso!
```

Nova base: Healthcare Fraud Detection

As células abaixo foram **adicionadas** ao teste, sem alterar as anteriores. Elas repetem a célula da base Credit Card com a **mesma configuração** (modelo, hiperparâmetros, sementes, split 60/20/20, limiar e métricas) na base `healthcare_fraud_detection.csv`, carregada direto do Google Drive (`/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/`). O rótulo de fraude é a coluna `Is_Fraud`.

1. A primeira célula registra os resultados da base Credit Card (célula acima) para o comparativo.
2. A segunda executa o teste na base healthcare.
3. A terceira compara as duas bases.

Engenharia de features da base healthcare (a mesma nos seis testes):

- removidos `Provider_ID` e `Claim_ID` (identificadores, como `nameOrig`/`nameDest` na PaySim) e `Claim_Submission_Date` (carimbo de tempo, como `Time` na Credit Card);
- `claim_amount_log` e `approved_amount_log` (log dos valores, como `amount_log`) e `approved_ratio` = valor aprovado ÷ valor solicitado (análoga ao `errorBalance` da PaySim);
- One-Hot das categóricas (`Patient_Gender`, `Diagnosis_Code`, `Procedure_Code`, `Insurance_Type`, `Provider_Specialty`, `Patient_State`, `Claim_Status`, `Visit_Type`), com ausentes na categoria `Desconhecido`;
- mediana para numéricas ausentes (`Prior_Visits_12m`).

```
# ======================================================================
# NOVA CÉLULA — registra os resultados da base Credit Card (célula acima)
# para o comparativo com a base Healthcare, feito na última célula.
# ======================================================================
import numpy as np

comparativo_bases = {}
try:
    # A base Credit Card tem 29 features após a engenharia (V1..V28 + amount_log)
    if X.shape[1] != 29:
        raise NameError('os resultados em memória não são da base Credit Card')
    comparativo_bases['Credit Card'] = {
        'Transações': int(len(y)), 'Taxa de fraude': float(np.mean(y)), 'Features': int(X.shape[1]),
        'ROC-AUC': float(roc_auc), 'PR-AUC': float(pr_auc), 'Precisão': float(prec), 'Recall': float(rec),
        'F1-Score': float(f1), 'Acurácia': float(acc), 'Throughput (TPS)': float(tps),
    }
    print("Resultados da base Credit Card registrados para o comparativo.")
except NameError as erro:
    print(f"Execute antes a célula da base Credit Card para incluí-la no comparativo ({erro}).")
```

```
Resultados da base Credit Card registrados para o comparativo.
```

```
# -*- coding: utf-8 -*-
"""
NOVA CÉLULA — Teste 1 na base Healthcare Fraud Detection (mesma configuração da célula Credit Card).
Base: healthcare_fraud_detection.csv (Google Drive) | Rótulo: Is_Fraud
Arquitetura: Simulação de Aprendizado Federado (Isolation Forest)
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import time
import os
import random

# Bibliotecas de Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import RobustScaler
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, average_precision_score, confusion_matrix,
                             roc_curve, precision_recall_curve)

# Fixar sementes para reprodutibilidade
def set_seed(seed=42):
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)

set_seed(42)

# Configurações de estilo
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)
```

```
print("\n--- 1. Carregamento dos Dados (Healthcare Fraud Detection) ---\n")
caminho_csv = '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/healthcare_fraud_detection.csv'
# Cópia da base no repositório, usada somente quando o Google Drive não está disponível (execução local)
caminhos_alternativos = ['PoC/base_de_dados/healthcare_fraud_detection.csv', 'base_de_dados/healthcare_fraud_detection.csv

try:
    from google.colab import drive
    drive.mount('/content/drive')
except ImportError:
    print("Fora do Google Colab: procurando a cópia da base no repositório.")

caminho = next((c for c in [caminho_csv] + caminhos_alternativos if os.path.exists(c)), None)
if caminho is None:
    raise FileNotFoundError(f"Base não encontrada: '{caminho_csv}'. Verifique se o Google Drive está montado.")
df = pd.read_csv(caminho)
print(f"Arquivo '{caminho}' carregado: {df.shape[0]} linhas, {df.shape[1]} colunas | "
      f"fraudes (Is_Fraud = 1): {int(df['Is_Fraud'].sum())} ({df['Is_Fraud'].mean():.2%})")
print("-" * 50)

print("\n--- 2. Limpeza de Colunas e Tratamento ---\n")

# Identificadores (Provider_ID, Claim_ID) não generalizam, como nameOrig/nameDest na PaySim, e a data de
# envio é um carimbo de tempo, como 'Time' na base Credit Card: as três colunas são removidas.
colunas_remover = ['Provider_ID', 'Claim_ID', 'Claim_Submission_Date']
df = df.drop(columns=[col for col in colunas_remover if col in df.columns])
df = df.dropna(subset=['Is_Fraud'])
df['Is_Fraud'] = df['Is_Fraud'].astype(int)
print(f"As colunas {colunas_remover} foram removidas (identificadores e data).")

print("\n--- 3. Criação de Novas Variáveis (Engenharia de Features) ---\n")

# Valores monetários com cauda longa -> log (como amount_log nas outras bases) e proporção aprovada
# pela operadora (análoga ao errorBalance da PaySim: sinistros fraudulentos têm aprovação bem menor)
df['approved_ratio'] = df['Approved_Amount'] / df['Claim_Amount'].replace(0, np.nan)
df['claim_amount_log'] = np.log1p(df['Claim_Amount'].clip(lower=0))
df['approved_amount_log'] = np.log1p(df['Approved_Amount'].clip(lower=0))
df = df.drop(columns=['Claim_Amount', 'Approved_Amount'])

# Categóricas -> One-Hot (Procedure_Code é um código, não uma quantidade); valor ausente vira 'Desconhecido'
colunas_categoricas = ['Patient_Gender', 'Diagnosis_Code', 'Procedure_Code', 'Insurance_Type',
                       'Provider_Specialty', 'Patient_State', 'Claim_Status', 'Visit_Type']
for col in colunas_categoricas:
    df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhecido').astype(str)
df = pd.get_dummies(df, columns=colunas_categoricas, drop_first=True, dtype=float)

# Numéricas ausentes (Prior_Visits_12m em 3,5% das linhas) -> mediana
for col in df.select_dtypes(include=np.number).columns:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].median())
print(f"Engenharia de features concluída: {df.shape[1] - 1} features.")
print("-" * 50)

print("\n--- 4. Preparando Dados para Machine Learning ---\n")

df_ml = df.copy()

# Separando X (Features) e y (Alvo)
X = df_ml.drop(columns=['Is_Fraud'])
y = df_ml['Is_Fraud']

# Divisão Correta: 60% Treino, 20% Validação, 20% Teste
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.25, random_state=42, stratify=y_temp)

print(f"Dados divididos! Treino: {X_train.shape[0]}, Validação: {X_val.shape[0]}, Teste: {X_test.shape[0]}")

# Escalonamento Robusto: Vital para o Isolation Forest não dar peso excessivo a valores anômalos
print("Aplicando RobustScaler para normalizar as grandezas numéricas...")
scaler = RobustScaler()
# O escalonador SÓ PODE ver os dados de treinamento
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

# Convertendo de volta para DataFrame por praticidade
X_train = pd.DataFrame(X_train_scaled, columns=X_train.columns)
X_val = pd.DataFrame(X_val_scaled, columns=X_val.columns)
X_test = pd.DataFrame(X_test_scaled, columns=X_test.columns)

print("\n--- 5. Simulação de Aprendizado Federado (FL) ---\n")
NUM_CLIENTES = 3
print(f"Simulando ambiente com {NUM_CLIENTES} clientes (silos de dados).")
```

```
# Divisão dos dados de treino entre os "clientes"
tamanho_chunk = len(X_train) // NUM_CLIENTES
clientes_X = [X_train.iloc[i * tamanho_chunk : (i + 1) * tamanho_chunk] for i in range(NUM_CLIENTES)]

modelos_locais = []
tempos_treinamento = []

print("\nIniciando Treinamento Local (Isolation Forest) nos clientes...")
for i in range(NUM_CLIENTES):
    inicio_treino = time.time()

    # IsolationForest OTIMIZADO:
    # Contaminação 'auto' permite que a floresta foque na estrutura intrínseca dos dados
    modelo_local = IsolationForest(
        n_estimators=150,
        max_samples=0.8,
        contamination='auto',
        n_jobs=-1,
        random_state=42+i
    )

    # Cliente treina apenas com sua fatia de dados
    modelo_local.fit(clientes_X[i])
    fim_treino = time.time()

    modelos_locais.append(modelo_local)
    tempos_treinamento.append(fim_treino - inicio_treino)
    print(f"-> Cliente {i+1} treinado em {fim_treino - inicio_treino:.2f} segundos.")

print("\n--- 6. Agregação Federada e Otimização de Limiar ---")

class ModeloFederado:
    def __init__(self, modelos_locais):
        self.modelos_locais = modelos_locais

    def decision_function(self, X):
        # Média dos escores de anomalia de todos os clientes locais
        escores = np.zeros(X.shape[0])
        for modelo in self.modelos_locais:
            escores += modelo.decision_function(X)
        return escores / len(self.modelos_locais)

modelo_global = ModeloFederado(modelos_locais)

# ESTRATÉGIA CORRIGIDA: Encontrar o melhor limiar no conjunto de VALIDAÇÃO
print("\nOtimizando o limiar de decisão via Curva PR no conjunto de VALIDAÇÃO (Sem Data Leakage)...")
# Invertemos o sinal: Valores MAIORES = maior probabilidade de fraude
y_scores_val = -modelo_global.decision_function(X_val)

# Calculando precisão e recall para vários limiares possíveis usando a Validação
precisions, recalls, thresholds = precision_recall_curve(y_val, y_scores_val)

# Calculando F1-Score para cada limiar (evitando divisão por zero)
f1_scores = np.divide(2 * (precisions * recalls), (precisions + recalls),
                      out=np.zeros_like(precisions), where=(precisions + recalls) != 0)

# O melhor limiar é aquele que maximiza o F1-Score na Validação
melhor_indice = np.argmax(f1_scores)
if melhor_indice < len(thresholds):
    limiar_otimizado = thresholds[melhor_indice]
else:
    # Prevenção contra vetor vazio
    limiar_otimizado = np.percentile(y_scores_val, 99)

print(f"Limiar matemático otimizado (F1 Máximo na Validação): {limiar_otimizado:.4f}")

print("\n--- 7. Inferência no Servidor Global (Teste Real) ---")
inicio_inferencia = time.time()

# Aplicando os escores estritamente na base de TESTE
y_scores_test = -modelo_global.decision_function(X_test)

# Classificando baseado no limiar otimizado, sem que o modelo tenha visto os dados
y_pred = (y_scores_test >= limiar_otimizado).astype(int)

fim_inferencia = time.time()
tempo_inferencia = fim_inferencia - inicio_inferencia

print("\n--- 8. Resultados Globais Pós-Otimização ---\n")

acc = accuracy_score(y_test, y_pred)
```

```
prec = precision_score(y_test, y_pred, zero_division=0)
rec = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
roc_auc = roc_auc_score(y_test, y_scores_test)
pr_auc = average_precision_score(y_test, y_scores_test)

print(">> MÉTRICAS DE QUALIDADE DO MODELO (ISOLATION FOREST):")
print(f"Acurácia Geral      : {acc * 100:.4f}%")
print(f"Precisão (Precision): {prec * 100:.4f}%")
print(f"Revocação (Recall)  : {rec * 100:.4f}%")
print(f"F1-Score            : {f1 * 100:.4f}%")
print(f"ROC-AUC             : {roc_auc:.4f}")
print(f"PR-AUC              : {pr_auc:.4f}")

tps = len(X_test) / tempo_inferencia if tempo_inferencia > 0 else float('inf')
print("\n>> MÉTRICAS DE RENDIMENTO TÉCNICO (THROUGHPUT):")
print(f"Amostras Processadas   : {len(X_test)} transações")
print(f"Tempo Total de Infe.   : {tempo_inferencia:.4f} segundos")
print(f"Rendimento (Throughput): {tps:.2f} transações por segundo\n")

# Visualizações
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 1. Matriz de Confusão (Aplicada sobre os dados de Teste)
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=['Normal', 'Fraude'], yticklabels=['Normal', 'Fraude'])
axes[0].set_title('Matriz de Confusão (Teste Final)')
axes[0].set_ylabel('Real')
axes[0].set_xlabel('Previsão')

# 2. Curva Precision-Recall (Do conjunto de Validação, onde o limiar foi escolhido)
axes[1].plot(recalls, precisions, color='purple', lw=2, label=f'PR-AUC (Val) = {average_precision_score(y_val, y_scores_val
# Proteção caso arrays estejam vazios ou melhor_indice fora dos limites
if len(recalls) > 0 and len(precisions) > 0 and melhor_indice < len(recalls):
    axes[1].scatter(recalls[melhor_indice], precisions[melhor_indice], color='red', s=100, label='Limiar Escolhido')
axes[1].set_xlabel('Recall (Revocação)')
axes[1].set_ylabel('Precision (Precisão)')
axes[1].set_title('Curva Precision-Recall (Dados de Validação)')
axes[1].legend(loc="lower left")

plt.tight_layout()
plt.show()

print("\nFim do processamento. Métricas robustas aplicadas com sucesso!")


comparativo_bases = globals().get('comparativo_bases', {})
comparativo_bases['Healthcare'] = {
    'Transações': int(len(y)), 'Taxa de fraude': float(np.mean(y)), 'Features': int(X.shape[1]),
    'ROC-AUC': float(roc_auc), 'PR-AUC': float(pr_auc), 'Precisão': float(prec), 'Recall': float(rec),
    'F1-Score': float(f1), 'Acurácia': float(acc), 'Throughput (TPS)': float(tps),
}
print("Resultados da base Healthcare registrados para o comparativo.")
```

```
--- 1. Carregamento dos Dados (Healthcare Fraud Detection) ---

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=
Arquivo '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/healthcare_fraud_detection.csv' carregado: 10000 linha
--------------------------------------------------

--- 2. Limpeza de Colunas e Tratamento ---

As colunas ['Provider_ID', 'Claim_ID', 'Claim_Submission_Date'] foram removidas (identificadores e data).

--- 3. Criação de Novas Variáveis (Engenharia de Features) ---

Engenharia de features concluída: 48 features.
--------------------------------------------------

--- 4. Preparando Dados para Machine Learning ---

Dados divididos! Treino: 6000, Validação: 2000, Teste: 2000
Aplicando RobustScaler para normalizar as grandezas numéricas...

--- 5. Simulação de Aprendizado Federado (FL) ---

Simulando ambiente com 3 clientes (silos de dados).

Iniciando Treinamento Local (Isolation Forest) nos clientes...
-> Cliente 1 treinado em 0.37 segundos.
-> Cliente 2 treinado em 0.38 segundos.
-> Cliente 3 treinado em 0.37 segundos.

--- 6. Agregação Federada e Otimização de Limiar ---

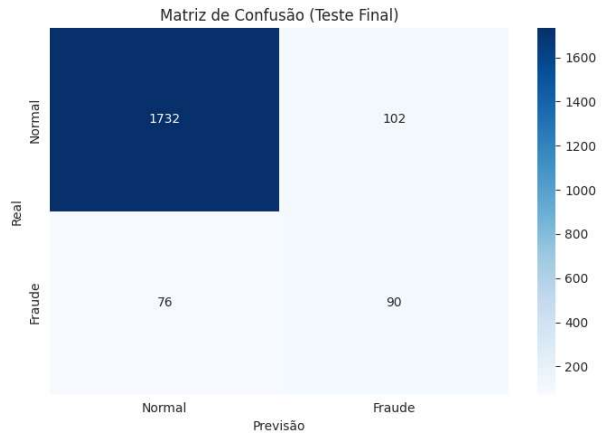
Otimizando o limiar de decisão via Curva PR no conjunto de VALIDAÇÃO (Sem Data Leakage)...
Limiar matemático otimizado (F1 Máximo na Validação): -0.0094

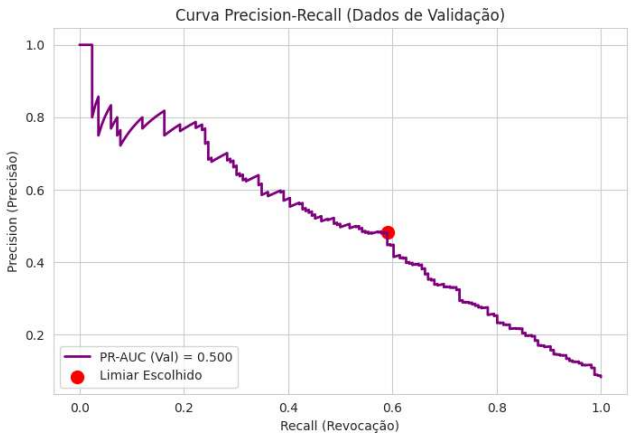
--- 7. Inferência no Servidor Global (Teste Real) ---

--- 8. Resultados Globais Pós-Otimização ---

>> MÉTRICAS DE QUALIDADE DO MODELO (ISOLATION FOREST):
Acurácia Geral      : 91.1000%
Precisão (Precision): 46.8750%
Revocação (Recall)  : 54.2169%
F1-Score            : 50.2793%
ROC-AUC             : 0.8702
PR-AUC              : 0.4976

>> MÉTRICAS DE RENDIMENTO TÉCNICO (THROUGHPUT):
Amostras Processadas   : 2000 transações
Tempo Total de Infe.   : 0.1295 segundos
Rendimento (Throughput): 15447.84 transações por segundo
```






```
Fim do processamento. Métricas robustas aplicadas com sucesso!
Resultados da base Healthcare registrados para o comparativo.
```

```
# ======================================================================
# NOVA CÉLULA — COMPARATIVO ENTRE AS BASES: Credit Card × Healthcare
# Teste 1 — Isolation Forest federado
# ======================================================================
import numpy as np
```

```
import pandas as pd
import matplotlib.pyplot as plt

# Cor fixa por base (paleta categórica validada para daltonismo): a mesma base tem sempre a mesma cor
CORES_BASES = {'Credit Card': '#2a78d6', 'Healthcare': '#eb6834', 'PaySim': '#1baf7a'}
METRICAS_COMP = ['ROC-AUC', 'PR-AUC', 'Precisão', 'Recall', 'F1-Score', 'Acurácia']
comparativo = pd.DataFrame(globals().get('comparativo_bases', {})).T

if comparativo.empty:
    print("Nenhum resultado registrado: execute as células das bases antes deste comparativo.")
else:
    # O PR-AUC de um classificador aleatório é igual à taxa de fraude. O PR-AUC normalizado desconta esse piso
    # (0 = aleatório, 1 = perfeito) e permite comparar bases com taxas de fraude diferentes.
    comparativo['PR-AUC normalizado'] = ((comparativo['PR-AUC'] - comparativo['Taxa de fraude'])
                                         / (1 - comparativo['Taxa de fraude']))

    print(">> CARACTERÍSTICAS DAS BASES")
    print(comparativo[['Transações', 'Taxa de fraude', 'Features']].to_string(formatters={
        'Transações': '{:,.0f}'.format, 'Taxa de fraude': '{:.2%}'.format, 'Features': '{:.0f}'.format}))

    print("\n>> MÉTRICAS NO CONJUNTO DE TESTE")
    print(comparativo[METRICAS_COMP + ['PR-AUC normalizado', 'Throughput (TPS)']].T.to_string(
        float_format=lambda v: f"{v:,.4f}"))

    if {'Credit Card', 'Healthcare'} <= set(comparativo.index):
        colunas_delta = METRICAS_COMP + ['PR-AUC normalizado']
        delta = comparativo.loc['Healthcare', colunas_delta] - comparativo.loc['Credit Card', colunas_delta]
        print("\n>> DIFERENÇA (Healthcare − Credit Card)")
        print(delta.map(lambda v: f"{v:+.4f}").to_string())

    print("\nObservação: o ROC-AUC não depende da taxa de fraude e é a métrica mais direta para comparar as bases. "
          "O PR-AUC depende dela (um classificador aleatório tem PR-AUC igual à taxa de fraude); por isso a "
          "comparação do PR-AUC entre bases deve usar o PR-AUC normalizado = (PR-AUC − taxa) / (1 − taxa).")

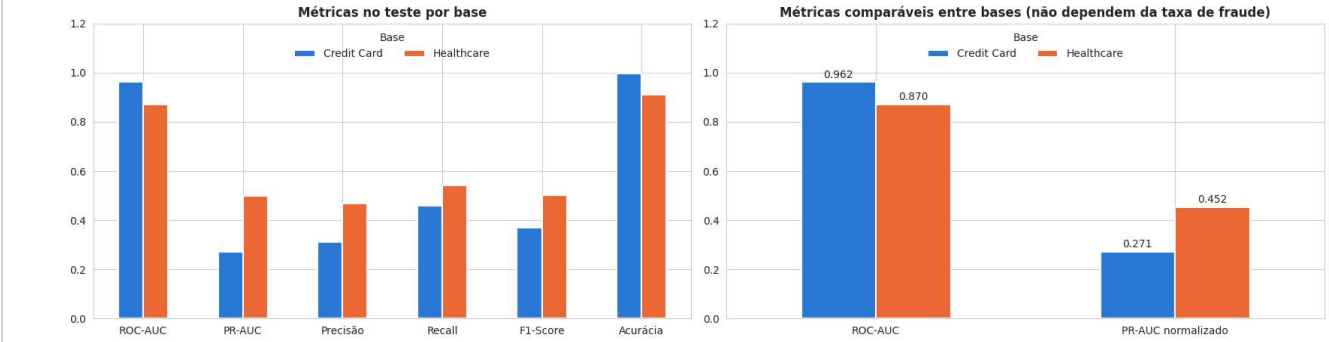
    fig, axes = plt.subplots(1, 2, figsize=(17, 5))
    cores = [CORES_BASES.get(b, '#7f7f7f') for b in comparativo.index]
    comparativo[METRICAS_COMP].T.plot.bar(ax=axes[0], rot=0, color=cores, edgecolor='white', linewidth=1.5)
    axes[0].set_ylim(0, 1.2)
    axes[0].set_title('Métricas no teste por base', fontweight='bold')
    axes[0].legend(title='Base', loc='upper center', ncol=len(comparativo), frameon=False)
    comparativo[['ROC-AUC', 'PR-AUC normalizado']].T.plot.bar(ax=axes[1], rot=0, color=cores,
                                                              edgecolor='white', linewidth=1.5)
    for barras in axes[1].containers:
        axes[1].bar_label(barras, fmt='%.3f', padding=2)
    axes[1].set_ylim(0, 1.2)
    axes[1].set_title('Métricas comparáveis entre bases (não dependem da taxa de fraude)', fontweight='bold')
    axes[1].legend(title='Base', loc='upper center', ncol=len(comparativo), frameon=False)
    plt.suptitle('Teste 1 — Isolation Forest federado', fontsize=13, fontweight='bold')
    plt.tight_layout()
    plt.show()
```

```
>> CARACTERÍSTICAS DAS BASES
            Transações Taxa de fraude Features
Credit Card    284,807          0.17%       29
Healthcare      10,000          8.29%       48

>> MÉTRICAS NO CONJUNTO DE TESTE
                   Credit Card  Healthcare
ROC-AUC                 0.9617      0.8702
PR-AUC                  0.2725      0.4976
Precisão                0.3103      0.4688
Recall                  0.4592      0.5422
F1-Score                0.3704      0.5028
Acurácia                0.9973      0.9110
PR-AUC normalizado      0.2712      0.4522
Throughput (TPS)    24,871.1626 15,447.8369

>> DIFERENÇA (Healthcare − Credit Card)
ROC-AUC               -0.0914
PR-AUC                +0.2252
Precisão              +0.1584
Recall                +0.0830
F1-Score              +0.1324
Acurácia              -0.0863
PR-AUC normalizado    +0.1810

Observação: o ROC-AUC não depende da taxa de fraude e é a métrica mais direta para comparar as bases. O PR-AUC depende dela
```



Novas bases: Bank Account Fraud (NeurIPS 2022), Insurance Claims e Fraud Oracle

As células abaixo foram **adicionadas** ao teste, sem alterar as anteriores. Elas repetem a célula da base Credit Card com a **mesma configuração** (modelo, hiperparâmetros, sementes, split 60/20/20, limiar e métricas) em mais três bases.

Origem das bases:

- **Bank Account Fraud Dataset Suite (NeurIPS 2022)** — só o arquivo `Base.csv` (213 MB, 1 milhão de solicitações de abertura de conta, ~1,1% de fraude), baixado com `kagglehub` (já instalado no Colab; localmente, `pip install kagglehub`): `kagglehub.dataset_download("sgpjesus/bank-account-fraud-dataset-neurips-2022", path="Base.csv")`. Sem acesso ao Kaggle, é usada a cópia do repositório, `PoC/base_de_dados/Base.csv.zip`.
- **Insurance Claims** — `insurance_claims.csv` (1.000 sinistros de seguro de automóvel, 24,7% de fraude), no repositório GitHub do projeto, em `PoC/base_de_dados/insurance_claims.csv`: a célula usa a cópia local quando o notebook roda a partir do repositório e, no Colab, lê o arquivo direto do GitHub.
- **Fraud Oracle** — `fraud_oracle.csv` (15.420 sinistros de seguro de veículos, 6,0% de fraude; rótulo `FraudFound_P`: 1 = fraude), no repositório GitHub do projeto, em `PoC/base_de_dados/fraud_oracle.csv`, lida da mesma forma que a Insurance Claims.

1. A primeira célula executa o teste na base BAF.
2. A segunda executa o teste na base Insurance Claims.
3. A terceira executa o teste na base Fraud Oracle.
4. A quarta compara as cinco bases (Credit Card, Healthcare, BAF, Insurance Claims e Fraud Oracle). Para incluir as duas primeiras, execute antes a célula da base Credit Card, a célula que registra os seus resultados e a célula da base Healthcare.

Engenharia de features (a mesma nos seis testes):

- **BAF** (rótulo `fraud_bool`): removidos `month` (mês da solicitação, carimbo de tempo como `Time` na Credit Card) e `device_fraud_count` (constante na `Base.csv`); nas seis colunas em que valor negativo significa ausente (`prev_address_months_count`, `current_address_months_count`, `intended_balcon_amount`, `bank_months_count`,

`session_length_in_minutes`, `device_distinct_emails_8w`), criado um indicador `_ausente` e o valor ausente vai para a mediana; log de `intended_balcon_amount`, `days_since_request`, `zip_count_4w` e `proposed_credit_limit`; One-Hot de `payment_type`, `employment_status`, `housing_status`, `source` e `device_os`.

- **Insurance Claims** (rótulo `fraud_reported`, Y/N → 1/0): removidos `policy_number`, `insured_zip` e `incident_location` (identificadores, como `nameOrig`/`nameDest` na PaySim), `_c39` (coluna vazia) e `auto_model` (39 modelos para 1.000 linhas; `auto_make` é mantida); as datas `policy_bind_date` e `incident_date` viram `dias_apolice_ate_incidente`; log de `total_claim_amount`, `injury_claim`, `property_claim` e `vehicle_claim`; One-Hot das 16 categóricas, com `?` na categoria `Desconhecido` e `authorities_contacted` ausente (texto `None` no CSV) como `Nenhuma`.
- **Fraud Oracle** (rótulo `FraudFound_P`): removidos `PolicyNumber` e `RepNumber` (identificadores), `Year` (carimbo de tempo, como `Time`) e `PolicyType` (repete `VehicleCategory` + `BasePolicy`); `Month` e `MonthClaimed` viram `meses_ate_sinistro`; as nove colunas de faixas em texto (`VehiclePrice`, `Days_Policy_Accident`, `Days_Policy_Claim`, `PastNumberOfClaims`, `AgeOfVehicle`, `AgeOfPolicyHolder`, `NumberOfSuppliments`, `AddressChange_Claim`, `NumberOfCars`) viram números em ordem crescente; `Age` = 0 (inválido) vai para a mediana; One-Hot das 12 categóricas, com `0` em `DayOfWeekClaimed` como `Desconhecido`.

```
# -*- coding: utf-8 -*-
"""
NOVA CÉLULA — Teste 1 na base Bank Account Fraud (NeurIPS 2022) — Base.csv (mesma configuração da célula Credit Card).
Base: sgpjesus/bank-account-fraud-dataset-neurips-2022, Base.csv (Kaggle, via kagglehub) | Rótulo: fraud_bool
Arquitetura: Simulação de Aprendizado Federado (Isolation Forest)
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import time
import os
import random

# Bibliotecas de Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import RobustScaler
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, average_precision_score, confusion_matrix,
                             roc_curve, precision_recall_curve)

# Fixar sementes para reprodutibilidade
def set_seed(seed=42):
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)

set_seed(42)

# Configurações de estilo
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)

print("\n--- 1. Carregamento dos Dados (Bank Account Fraud (NeurIPS 2022) — Base.csv) ---\n")
caminho = None
try:
    import kagglehub
    # Download latest version — apenas o arquivo Base.csv (213 MB) do Bank Account Fraud Dataset Suite
    path = kagglehub.dataset_download("sgpjesus/bank-account-fraud-dataset-neurips-2022", path="Base.csv")
    print("Path to dataset files:", path)
    caminho = path
except Exception as erro:
    print(f"Download pelo kagglehub indisponível ({type(erro).__name__}): procurando uma cópia local da base.")
if caminho is None or not os.path.exists(caminho):
    # Cópia no repositório (zip lido direto pelo pandas); executando da raiz do repositório ou de dentro de PoC/
    caminho = next((c for c in ['PoC/base_de_dados/Base.csv.zip', 'base_de_dados/Base.csv.zip',
                                'PoC/base_de_dados/Base.csv', 'base_de_dados/Base.csv'] if os.path.exists(c)), None)
if caminho is None:
    raise FileNotFoundError("Base.csv (Bank Account Fraud) não encontrada: verifique o acesso ao Kaggle (kagglehub) "
                            "ou copie o arquivo para PoC/base_de_dados/.")
df = pd.read_csv(caminho)
print(f"Arquivo '{caminho}' carregado: {df.shape[0]} linhas, {df.shape[1]} colunas | "
      f"fraudes (fraud_bool = 1): {int(df['fraud_bool'].sum())} ({df['fraud_bool'].mean():.2%})")
print("-" * 50)

print("\n--- 2. Limpeza de Colunas e Tratamento ---\n")

# 'month' é o mês da solicitação (carimbo de tempo, como 'Time' na base Credit Card) e 'device_fraud_count' é
# constante (zero) na Base.csv: as duas colunas são removidas.
df = df.drop(columns=['fraud_bool'])
```

```
df = df.dropna(subset=['fraud_bool'])
df['fraud_bool'] = df['fraud_bool'].astype(int)
colunas_remover = ['month', 'device_fraud_count']
df = df.drop(columns=[col for col in colunas_remover if col in df.columns])
print(f"As colunas {colunas_remover} foram removidas (tempo e coluna constante).")

print("\n--- 3. Criação de Novas Variáveis (Engenharia de Features) ---\n")

# Na BAF, valores negativos nestas colunas codificam "ausente" (documentação do dataset): cada uma ganha um
# indicador de ausência e o valor ausente vai para a mediana (abaixo)
colunas_ausente_negativo = ['prev_address_months_count', 'current_address_months_count', 'intended_balcon_amount',
                            'bank_months_count', 'session_length_in_minutes', 'device_distinct_emails_8w']
for col in colunas_ausente_negativo:
    df[col + '_ausente'] = (df[col] < 0).astype(float)
    df[col] = df[col].where(df[col] >= 0)

# Contagens e valores com cauda longa -> log (como amount_log nas outras bases)
for col in ['intended_balcon_amount', 'days_since_request', 'zip_count_4w', 'proposed_credit_limit']:
    df[col + '_log'] = np.log1p(df[col].clip(lower=0))
df = df.drop(columns=['intended_balcon_amount', 'days_since_request', 'zip_count_4w', 'proposed_credit_limit'])

# Categóricas -> One-Hot; valor ausente vira 'Desconhecido'
colunas_categoricas = ['payment_type', 'employment_status', 'housing_status', 'source', 'device_os']
for col in colunas_categoricas:
    df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhecido').astype(str)
df = pd.get_dummies(df, columns=colunas_categoricas, drop_first=True, dtype=float)

# Numéricas ausentes -> mediana
for col in df.select_dtypes(include=np.number).columns:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].median())
print(f"Engenharia de features concluída: {df.shape[1] - 1} features.")
print("-" * 50)

print("\n--- 4. Preparando Dados para Machine Learning ---\n")

df_ml = df.copy()

# Separando X (Features) e y (Alvo)
X = df_ml.drop(columns=['fraud_bool'])
y = df_ml['fraud_bool']

# Divisão Correta: 60% Treino, 20% Validação, 20% Teste
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.25, random_state=42, stratify=y_temp)

print(f"Dados divididos! Treino: {X_train.shape[0]}, Validação: {X_val.shape[0]}, Teste: {X_test.shape[0]}")

# Escalonamento Robusto: Vital para o Isolation Forest não dar peso excessivo a valores anômalos
print("Aplicando RobustScaler para normalizar as grandezas numéricas...")
scaler = RobustScaler()
# O escalonador SÓ PODE ver os dados de treinamento
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

# Convertendo de volta para DataFrame por praticidade
X_train = pd.DataFrame(X_train_scaled, columns=X_train.columns)
X_val = pd.DataFrame(X_val_scaled, columns=X_val.columns)
X_test = pd.DataFrame(X_test_scaled, columns=X_test.columns)

print("\n--- 5. Simulação de Aprendizado Federado (FL) ---\n")
NUM_CLIENTES = 3
print(f"Simulando ambiente com {NUM_CLIENTES} clientes (silos de dados).")

# Divisão dos dados de treino entre os "clientes"
tamanho_chunk = len(X_train) // NUM_CLIENTES
clientes_X = [X_train.iloc[i * tamanho_chunk : (i + 1) * tamanho_chunk] for i in range(NUM_CLIENTES)]

modelos_locais = []
tempos_treinamento = []

print("\nIniciando Treinamento Local (Isolation Forest) nos clientes...")
for i in range(NUM_CLIENTES):
    inicio_treino = time.time()

    # IsolationForest OTIMIZADO:
    # Contaminação 'auto' permite que a floresta foque na estrutura intrínseca dos dados
    modelo_local = IsolationForest(
        n_estimators=150,
        max_samples=0.8,
        contamination='auto',
        n_jobs=-1,
```

```
        random_state=42+i
    )

    # Cliente treina apenas com sua fatia de dados
    modelo_local.fit(clientes_X[i])
    fim_treino = time.time()

    modelos_locais.append(modelo_local)
    tempos_treinamento.append(fim_treino - inicio_treino)
    print(f"-> Cliente {i+1} treinado em {fim_treino - inicio_treino:.2f} segundos.")

print("\n--- 6. Agregação Federada e Otimização de Limiar ---")

class ModeloFederado:
    def __init__(self, modelos_locais):
        self.modelos_locais = modelos_locais

    def decision_function(self, X):
        # Média dos escores de anomalia de todos os clientes locais
        escores = np.zeros(X.shape[0])
        for modelo in self.modelos_locais:
            escores += modelo.decision_function(X)
        return escores / len(self.modelos_locais)

modelo_global = ModeloFederado(modelos_locais)

# ESTRATÉGIA CORRIGIDA: Encontrar o melhor limiar no conjunto de VALIDAÇÃO
print("\nOtimizando o limiar de decisão via Curva PR no conjunto de VALIDAÇÃO (Sem Data Leakage)...")
# Invertemos o sinal: Valores MAIORES = maior probabilidade de fraude
y_scores_val = -modelo_global.decision_function(X_val)

# Calculando precisão e recall para vários limiares possíveis usando a Validação
precisions, recalls, thresholds = precision_recall_curve(y_val, y_scores_val)

# Calculando F1-Score para cada limiar (evitando divisão por zero)
f1_scores = np.divide(2 * (precisions * recalls), (precisions + recalls),
                      out=np.zeros_like(precisions), where=(precisions + recalls) != 0)

# O melhor limiar é aquele que maximiza o F1-Score na Validação
melhor_indice = np.argmax(f1_scores)
if melhor_indice < len(thresholds):
    limiar_otimizado = thresholds[melhor_indice]
else:
    # Prevenção contra vetor vazio
    limiar_otimizado = np.percentile(y_scores_val, 99)

print(f"Limiar matemático otimizado (F1 Máximo na Validação): {limiar_otimizado:.4f}")

print("\n--- 7. Inferência no Servidor Global (Teste Real) ---")
inicio_inferencia = time.time()

# Aplicando os escores estritamente na base de TESTE
y_scores_test = -modelo_global.decision_function(X_test)

# Classificando baseado no limiar otimizado, sem que o modelo tenha visto os dados
y_pred = (y_scores_test >= limiar_otimizado).astype(int)

fim_inferencia = time.time()
tempo_inferencia = fim_inferencia - inicio_inferencia

print("\n--- 8. Resultados Globais Pós-Otimização ---\n")

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, zero_division=0)
rec = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
roc_auc = roc_auc_score(y_test, y_scores_test)
pr_auc = average_precision_score(y_test, y_scores_test)

print(">> MÉTRICAS DE QUALIDADE DO MODELO (ISOLATION FOREST):")
print(f"Acurácia Geral      : {acc * 100:.4f}%")
print(f"Precisão (Precision): {prec * 100:.4f}%")
print(f"Revocação (Recall)  : {rec * 100:.4f}%")
print(f"F1-Score            : {f1 * 100:.4f}%")
print(f"ROC-AUC             : {roc_auc:.4f}")
print(f"PR-AUC              : {pr_auc:.4f}")

tps = len(X_test) / tempo_inferencia if tempo_inferencia > 0 else float('inf')
print("\n>> MÉTRICAS DE RENDIMENTO TÉCNICO (THROUGHPUT):")
print(f"Amostras Processadas   : {len(X_test)} transações")
print(f"Tempo Total de Infe.   : {tempo_inferencia:.4f} segundos")
print(f"Rendimento (Throughput): {tps:.2f} transações por segundo\n")
```

```
# Visualizações
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 1. Matriz de Confusão (Aplicada sobre os dados de Teste)
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=['Normal', 'Fraude'], yticklabels=['Normal', 'Fraude'])
axes[0].set_title('Matriz de Confusão (Teste Final)')
axes[0].set_ylabel('Real')
axes[0].set_xlabel('Previsão')

# 2. Curva Precision-Recall (Do conjunto de Validação, onde o limiar foi escolhido)
axes[1].plot(recalls, precisions, color='purple', lw=2, label=f'PR-AUC (Val) = {average_precision_score(y_val, y_scores_val)
# Proteção caso arrays estejam vazios ou melhor_indice fora dos limites
if len(recalls) > 0 and len(precisions) > 0 and melhor_indice < len(recalls):
    axes[1].scatter(recalls[melhor_indice], precisions[melhor_indice], color='red', s=100, label='Limiar Escolhido')
axes[1].set_xlabel('Recall (Revocação)')
axes[1].set_ylabel('Precision (Precisão)')
axes[1].set_title('Curva Precision-Recall (Dados de Validação)')
axes[1].legend(loc="lower left")

plt.tight_layout()
plt.show()

print("\nFim do processamento. Métricas robustas aplicadas com sucesso!")


comparativo_bases = globals().get('comparativo_bases', {})
comparativo_bases['BAF'] = {
    'Transações': int(len(y)), 'Taxa de fraude': float(np.mean(y)), 'Features': int(X.shape[1]),
    'ROC-AUC': float(roc_auc), 'PR-AUC': float(pr_auc), 'Precisão': float(prec), 'Recall': float(rec),
    'F1-Score': float(f1), 'Acurácia': float(acc), 'Throughput (TPS)': float(tps),
}
print("Resultados da base BAF registrados para o comparativo.")
```

```
--- 1. Carregamento dos Dados (Bank Account Fraud (NeurIPS 2022) — Base.csv) ---

Downloading to /root/.cache/kagglehub/datasets/sgpjesus/bank-account-fraud-dataset-neurips-2022/versions/2/Base.csv...
100%|██████████| 82.4M/82.4M [00:00<00:00, 153MB/s]Extracting zip of Base.csv...

Path to dataset files: /root/.cache/kagglehub/datasets/sgpjesus/bank-account-fraud-dataset-neurips-2022/versions/2/Base.cs
Arquivo '/root/.cache/kagglehub/datasets/sgpjesus/bank-account-fraud-dataset-neurips-2022/versions/2/Base.csv' carregado:
--------------------------------------------------

--- 2. Limpeza de Colunas e Tratamento ---

As colunas ['month', 'device_fraud_count'] foram removidas (tempo e coluna constante).

--- 3. Criação de Novas Variáveis (Engenharia de Features) ---

Engenharia de features concluída: 51 features.
--------------------------------------------------

--- 4. Preparando Dados para Machine Learning ---

Dados divididos! Treino: 600000, Validação: 200000, Teste: 200000
Aplicando RobustScaler para normalizar as grandezas numéricas...

--- 5. Simulação de Aprendizado Federado (FL) ---

Simulando ambiente com 3 clientes (silos de dados).

Iniciando Treinamento Local (Isolation Forest) nos clientes...
-> Cliente 1 treinado em 10.80 segundos.
-> Cliente 2 treinado em 16.21 segundos.
-> Cliente 3 treinado em 12.92 segundos.

--- 6. Agregação Federada e Otimização de Limiar ---

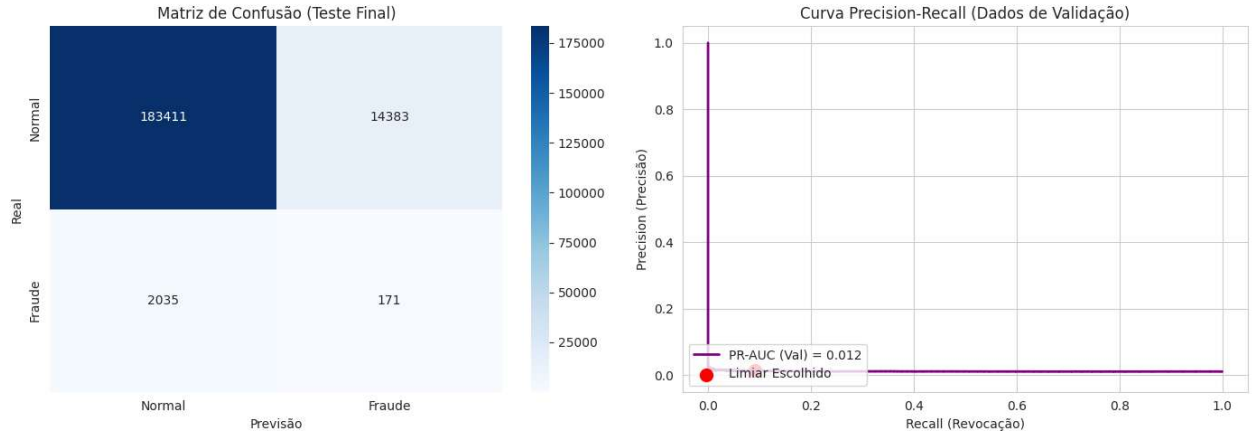
Otimizando o limiar de decisão via Curva PR no conjunto de VALIDAÇÃO (Sem Data Leakage)...
Limiar matemático otimizado (F1 Máximo na Validação): -0.0432

--- 7. Inferência no Servidor Global (Teste Real) ---

--- 8. Resultados Globais Pós-Otimização ---

>> MÉTRICAS DE QUALIDADE DO MODELO (ISOLATION FOREST):
Acurácia Geral      : 91.7910%
Precisão (Precision): 1.1749%
Revocação (Recall)  : 7.7516%
F1-Score            : 2.0406%
ROC-AUC             : 0.5035
PR-AUC              : 0.0113

>> MÉTRICAS DE RENDIMENTO TÉCNICO (THROUGHPUT):
Amostras Processadas   : 200000 transações
Tempo Total de Infe.   : 17.1457 segundos
Rendimento (Throughput): 11664.76 transações por segundo
```




```
Fim do processamento. Métricas robustas aplicadas com sucesso!
Resultados da base BAF registrados para o comparativo.
```

```
# -*- coding: utf-8 -*-
"""
```

```
NOVA CÉLULA — Teste 1 na base Insurance Claims (sinistros de seguro de automóvel) (mesma configuração da célula Credit Card)
Base: insurance_claims.csv (repositório GitHub, PoC/base_de_dados/) | Rótulo: fraud_reported
Arquitetura: Simulação de Aprendizado Federado (Isolation Forest)
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import time
import os
import random

# Bibliotecas de Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import RobustScaler
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, average_precision_score, confusion_matrix,
                             roc_curve, precision_recall_curve)

# Fixar sementes para reprodutibilidade
def set_seed(seed=42):
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)

set_seed(42)

# Configurações de estilo
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)

print("\n--- 1. Carregamento dos Dados (Insurance Claims (sinistros de seguro de automóvel)) ---\n")
# Base no repositório GitHub do projeto (PoC/base_de_dados/insurance_claims.csv): usa a cópia local quando o
# notebook roda a partir do repositório; no Colab, lê o arquivo direto do GitHub
URL_GITHUB = 'https://raw.githubusercontent.com/suxlcmd/MPES_Lucas_Cerqueira/main/PoC/base_de_dados/insurance_claims.csv'
caminho = next((c for c in ['PoC/base_de_dados/insurance_claims.csv', 'base_de_dados/insurance_claims.csv']
                if os.path.exists(c)), URL_GITHUB)
df = pd.read_csv(caminho)
print(f"Arquivo '{caminho}' carregado: {df.shape[0]} linhas, {df.shape[1]} colunas | "
      f"fraudes (fraud_reported = Y): {int((df['fraud_reported'] == 'Y').sum())} ({(df['fraud_reported'] == 'Y').mean():.2%}
print("-" * 50)

print("\n--- 2. Limpeza de Colunas e Tratamento ---\n")

# Rótulo Y/N -> 1/0. Identificadores e texto único por linha (policy_number, insured_zip, incident_location) não
# generalizam, como nameOrig/nameDest na PaySim; _c39 é uma coluna vazia; auto_model tem 39 modelos para 1.000 linhas
# (auto_make é mantida). As duas datas viram um único intervalo: dias entre a contratação da apólice e o incidente.
df['fraud_reported'] = df['fraud_reported'].astype(str).str.strip().str.upper().map({'Y': 1, 'N': 0})
df = df.dropna(subset=['fraud_reported'])
df['fraud_reported'] = df['fraud_reported'].astype(int)
df['dias_apolice_ate_incidente'] = (pd.to_datetime(df['incident_date'], errors='coerce')
                                    - pd.to_datetime(df['policy_bind_date'], errors='coerce')).dt.days
colunas_remover = ['policy_number', 'insured_zip', 'incident_location', '_c39', 'auto_model',
                   'policy_bind_date', 'incident_date']
df = df.drop(columns=[col for col in colunas_remover if col in df.columns])
print(f"As colunas {colunas_remover} foram removidas (identificadores, coluna vazia, alta cardinalidade e datas).")

print("\n--- 3. Criação de Novas Variáveis (Engenharia de Features) ---\n")

# Valores do sinistro com cauda longa -> log (como amount_log nas outras bases)
for col in ['total_claim_amount', 'injury_claim', 'property_claim', 'vehicle_claim']:
    df[col + '_log'] = np.log1p(df[col].clip(lower=0))
df = df.drop(columns=['total_claim_amount', 'injury_claim', 'property_claim', 'vehicle_claim'])

# Categóricas -> One-Hot. '?' marca valor desconhecido; em authorities_contacted, o texto 'None' (nenhuma autoridade
# acionada) é lido pelo pandas como ausente e vira 'Nenhuma'
df['authorities_contacted'] = df['authorities_contacted'].fillna('Nenhuma')
colunas_categoricas = ['policy_state', 'policy_csl', 'insured_sex', 'insured_education_level', 'insured_occupation',
                       'insured_hobbies', 'insured_relationship', 'incident_type', 'collision_type', 'incident_severity',
                       'authorities_contacted', 'incident_state', 'incident_city', 'property_damage',
                       'police_report_available', 'auto_make']
for col in colunas_categoricas:
    df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhecido').astype(str).replace('?', 'Desconhecido')
df = pd.get_dummies(df, columns=colunas_categoricas, drop_first=True, dtype=float)

# Numéricas ausentes -> mediana
for col in df.select_dtypes(include=np.number).columns:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].median())
print(f"Engenharia de features concluída: {df.shape[1] - 1} features.")
print("-" * 50)
```

```
print("\n--- 4. Preparando Dados para Machine Learning ---\n")

df_ml = df.copy()

# Separando X (Features) e y (Alvo)
X = df_ml.drop(columns=['fraud_reported'])
y = df_ml['fraud_reported']

# Divisão Correta: 60% Treino, 20% Validação, 20% Teste
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.25, random_state=42, stratify=y_temp)

print(f"Dados divididos! Treino: {X_train.shape[0]}, Validação: {X_val.shape[0]}, Teste: {X_test.shape[0]}")

# Escalonamento Robusto: Vital para o Isolation Forest não dar peso excessivo a valores anômalos
print("Aplicando RobustScaler para normalizar as grandezas numéricas...")
scaler = RobustScaler()
# O escalonador SÓ PODE ver os dados de treinamento
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

# Convertendo de volta para DataFrame por praticidade
X_train = pd.DataFrame(X_train_scaled, columns=X_train.columns)
X_val = pd.DataFrame(X_val_scaled, columns=X_val.columns)
X_test = pd.DataFrame(X_test_scaled, columns=X_test.columns)

print("\n--- 5. Simulação de Aprendizado Federado (FL) ---\n")
NUM_CLIENTES = 3
print(f"Simulando ambiente com {NUM_CLIENTES} clientes (silos de dados).")

# Divisão dos dados de treino entre os "clientes"
tamanho_chunk = len(X_train) // NUM_CLIENTES
clientes_X = [X_train.iloc[i * tamanho_chunk : (i + 1) * tamanho_chunk] for i in range(NUM_CLIENTES)]

modelos_locais = []
tempos_treinamento = []

print("\nIniciando Treinamento Local (Isolation Forest) nos clientes...")
for i in range(NUM_CLIENTES):
    inicio_treino = time.time()

    # IsolationForest OTIMIZADO:
    # Contaminação 'auto' permite que a floresta foque na estrutura intrínseca dos dados
    modelo_local = IsolationForest(
        n_estimators=150,
        max_samples=0.8,
        contamination='auto',
        n_jobs=-1,
        random_state=42+i
    )

    # Cliente treina apenas com sua fatia de dados
    modelo_local.fit(clientes_X[i])
    fim_treino = time.time()

    modelos_locais.append(modelo_local)
    tempos_treinamento.append(fim_treino - inicio_treino)
    print(f"-> Cliente {i+1} treinado em {fim_treino - inicio_treino:.2f} segundos.")

print("\n--- 6. Agregação Federada e Otimização de Limiar ---")

class ModeloFederado:
    def __init__(self, modelos_locais):
        self.modelos_locais = modelos_locais

    def decision_function(self, X):
        # Média dos escores de anomalia de todos os clientes locais
        escores = np.zeros(X.shape[0])
        for modelo in self.modelos_locais:
            escores += modelo.decision_function(X)
        return escores / len(self.modelos_locais)

modelo_global = ModeloFederado(modelos_locais)

# ESTRATÉGIA CORRIGIDA: Encontrar o melhor limiar no conjunto de VALIDAÇÃO
print("\nOtimizando o limiar de decisão via Curva PR no conjunto de VALIDAÇÃO (Sem Data Leakage)...")
# Invertemos o sinal: Valores MAIORES = maior probabilidade de fraude
y_scores_val = -modelo_global.decision_function(X_val)

# Calculando precisão e recall para vários limiares possíveis usando a Validação
```

```
precisions, recalls, thresholds = precision_recall_curve(y_val, y_scores_val)

# Calculando F1-Score para cada limiar (evitando divisão por zero)
f1_scores = np.divide(2 * (precisions * recalls), (precisions + recalls),
                      out=np.zeros_like(precisions), where=(precisions + recalls) != 0)

# O melhor limiar é aquele que maximiza o F1-Score na Validação
melhor_indice = np.argmax(f1_scores)
if melhor_indice < len(thresholds):
    limiar_otimizado = thresholds[melhor_indice]
else:
    # Prevenção contra vetor vazio
    limiar_otimizado = np.percentile(y_scores_val, 99)

print(f"Limiar matemático otimizado (F1 Máximo na Validação): {limiar_otimizado:.4f}")

print("\n--- 7. Inferência no Servidor Global (Teste Real) ---")
inicio_inferencia = time.time()

# Aplicando os escores estritamente na base de TESTE
y_scores_test = -modelo_global.decision_function(X_test)

# Classificando baseado no limiar otimizado, sem que o modelo tenha visto os dados
y_pred = (y_scores_test >= limiar_otimizado).astype(int)

fim_inferencia = time.time()
tempo_inferencia = fim_inferencia - inicio_inferencia

print("\n--- 8. Resultados Globais Pós-Otimização ---\n")

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, zero_division=0)
rec = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
roc_auc = roc_auc_score(y_test, y_scores_test)
pr_auc = average_precision_score(y_test, y_scores_test)

print(">> MÉTRICAS DE QUALIDADE DO MODELO (ISOLATION FOREST):")
print(f"Acurácia Geral      : {acc * 100:.4f}%")
print(f"Precisão (Precision): {prec * 100:.4f}%")
print(f"Revocação (Recall)  : {rec * 100:.4f}%")
print(f"F1-Score            : {f1 * 100:.4f}%")
print(f"ROC-AUC             : {roc_auc:.4f}")
print(f"PR-AUC              : {pr_auc:.4f}")

tps = len(X_test) / tempo_inferencia if tempo_inferencia > 0 else float('inf')
print("\n>> MÉTRICAS DE RENDIMENTO TÉCNICO (THROUGHPUT):")
print(f"Amostras Processadas   : {len(X_test)} transações")
print(f"Tempo Total de Infe.   : {tempo_inferencia:.4f} segundos")
print(f"Rendimento (Throughput): {tps:.2f} transações por segundo\n")

# Visualizações
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 1. Matriz de Confusão (Aplicada sobre os dados de Teste)
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=['Normal', 'Fraude'], yticklabels=['Normal', 'Fraude'])
axes[0].set_title('Matriz de Confusão (Teste Final)')
axes[0].set_ylabel('Real')
axes[0].set_xlabel('Previsão')

# 2. Curva Precision-Recall (Do conjunto de Validação, onde o limiar foi escolhido)
axes[1].plot(recalls, precisions, color='purple', lw=2, label=f'PR-AUC (Val) = {average_precision_score(y_val, y_scores_val)
# Proteção caso arrays estejam vazios ou melhor_indice fora dos limites
if len(recalls) > 0 and len(precisions) > 0 and melhor_indice < len(recalls):
    axes[1].scatter(recalls[melhor_indice], precisions[melhor_indice], color='red', s=100, label='Limiar Escolhido')
axes[1].set_xlabel('Recall (Revocação)')
axes[1].set_ylabel('Precision (Precisão)')
axes[1].set_title('Curva Precision-Recall (Dados de Validação)')
axes[1].legend(loc="lower left")

plt.tight_layout()
plt.show()

print("\nFim do processamento. Métricas robustas aplicadas com sucesso!")


comparativo_bases = globals().get('comparativo_bases', {})
comparativo_bases['Insurance Claims'] = {
    'Transações': int(len(y)), 'Taxa de fraude': float(np.mean(y)), 'Features': int(X.shape[1]),
    'ROC-AUC': float(roc_auc), 'PR-AUC': float(pr_auc), 'Precisão': float(prec), 'Recall': float(rec),
    'F1-Score': float(f1), 'Acurácia': float(acc), 'Throughput (TPS)': float(tps),
```

```
}
print("Resultados da base Insurance Claims registrados para o comparativo.")
```

```
--- 1. Carregamento dos Dados (Insurance Claims (sinistros de seguro de automóvel)) ---

Arquivo 'https://raw.githubusercontent.com/suxlcmd/MPES_Lucas_Cerqueira/main/PoC/base_de_dados/insurance_claims.csv' carre
--------------------------------------------------

--- 2. Limpeza de Colunas e Tratamento ---

As colunas ['policy_number', 'insured_zip', 'incident_location', '_c39', 'auto_model', 'policy_bind_date', 'incident_date'

--- 3. Criação de Novas Variáveis (Engenharia de Features) ---

Engenharia de features concluída: 107 features.
--------------------------------------------------

--- 4. Preparando Dados para Machine Learning ---

Dados divididos! Treino: 600, Validação: 200, Teste: 200
Aplicando RobustScaler para normalizar as grandezas numéricas...

--- 5. Simulação de Aprendizado Federado (FL) ---

Simulando ambiente com 3 clientes (silos de dados).

Iniciando Treinamento Local (Isolation Forest) nos clientes...
-> Cliente 1 treinado em 0.36 segundos.
-> Cliente 2 treinado em 0.41 segundos.
-> Cliente 3 treinado em 0.40 segundos.

--- 6. Agregação Federada e Otimização de Limiar ---

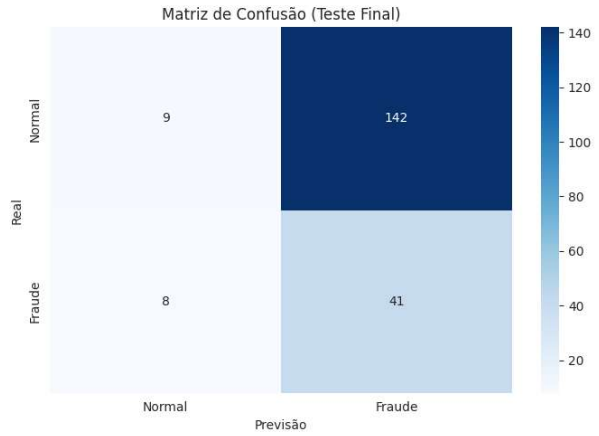
Otimizando o limiar de decisão via Curva PR no conjunto de VALIDAÇÃO (Sem Data Leakage)...
Limiar matemático otimizado (F1 Máximo na Validação): -0.0705

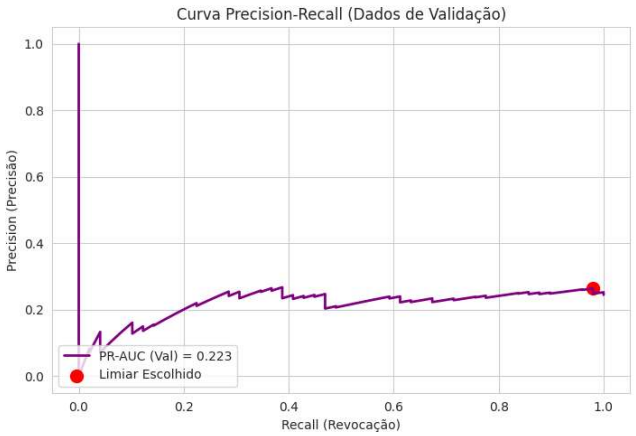
--- 7. Inferência no Servidor Global (Teste Real) ---

--- 8. Resultados Globais Pós-Otimização ---

>> MÉTRICAS DE QUALIDADE DO MODELO (ISOLATION FOREST):
Acurácia Geral      : 25.0000%
Precisão (Precision): 22.4044%
Revocação (Recall)  : 83.6735%
F1-Score            : 35.3448%
ROC-AUC             : 0.4238
PR-AUC              : 0.2137

>> MÉTRICAS DE RENDIMENTO TÉCNICO (THROUGHPUT):
Amostras Processadas   : 200 transações
Tempo Total de Infe.   : 0.0658 segundos
Rendimento (Throughput): 3039.13 transações por segundo
```






```
Fim do processamento. Métricas robustas aplicadas com sucesso!
Resultados da base Insurance Claims registrados para o comparativo.
```

```
# -*- coding: utf-8 -*-
"""
NOVA CÉLULA — Teste 1 na base Fraud Oracle (fraude em sinistros de seguro de veículos) (mesma configuração da célula Cred
Base: fraud_oracle.csv (repositório GitHub, PoC/base_de_dados/) | Rótulo: FraudFound_P
Arquitetura: Simulação de Aprendizado Federado (Isolation Forest)
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import time
import os
import random

# Bibliotecas de Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import RobustScaler
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, average_precision_score, confusion_matrix,
                             roc_curve, precision_recall_curve)

# Fixar sementes para reprodutibilidade
def set_seed(seed=42):
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)

set_seed(42)

# Configurações de estilo
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)

print("\n--- 1. Carregamento dos Dados (Fraud Oracle (fraude em sinistros de seguro de veículos)) ---\n")
# Base no repositório GitHub do projeto (PoC/base_de_dados/fraud_oracle.csv): usa a cópia local quando o
# notebook roda a partir do repositório; no Colab, lê o arquivo direto do GitHub
URL_GITHUB = 'https://raw.githubusercontent.com/suxlcmd/MPES_Lucas_Cerqueira/main/PoC/base_de_dados/fraud_oracle.csv'
caminho = next((c for c in ['PoC/base_de_dados/fraud_oracle.csv', 'base_de_dados/fraud_oracle.csv']
                if os.path.exists(c)), URL_GITHUB)
df = pd.read_csv(caminho)
print(f"Arquivo '{caminho}' carregado: {df.shape[0]} linhas, {df.shape[1]} colunas | "
      f"fraudes (FraudFound_P = 1): {int(df['FraudFound_P'].sum())} ({df['FraudFound_P'].mean():.2%})")
print("-" * 50)

print("\n--- 2. Limpeza de Colunas e Tratamento ---\n")

df = df.dropna(subset=['FraudFound_P'])
df['FraudFound_P'] = df['FraudFound_P'].astype(int)
# PolicyNumber (sequencial) e RepNumber (representante) são identificadores, como nameOrig/nameDest na PaySim;
# Year é carimbo de tempo, como 'Time' na base Credit Card; PolicyType repete VehicleCategory + BasePolicy.
# Month e MonthClaimed viram um único intervalo: meses entre o acidente e a abertura do sinistro.
MESES = {'Jan': 1, 'Feb': 2, 'Mar': 3, 'Apr': 4, 'May': 5, 'Jun': 6, 'Jul': 7, 'Aug': 8, 'Sep': 9, 'Oct': 10, 'Nov': 11,
df['meses_ate_sinistro'] = (df['MonthClaimed'].map(MESES) - df['Month'].map(MESES)) % 12
colunas_remover = ['PolicyNumber', 'RepNumber', 'Year', 'PolicyType', 'Month', 'MonthClaimed']
df = df.drop(columns=[col for col in colunas_remover if col in df.columns])
print(f"As colunas {colunas_remover} foram removidas (identificadores, ano, coluna redundante e meses).")

print("\n--- 3. Criação de Novas Variáveis (Engenharia de Features) ---\n")

# Faixas de texto -> números em ordem crescente (preserva a ordem e evita dezenas de colunas One-Hot)
FAIXAS_ORDINAIS = {
    'VehiclePrice': ['less than 20000', '20000 to 29000', '30000 to 39000', '40000 to 59000', '60000 to 69000', 'more tha
    'Days_Policy_Accident': ['none', '1 to 7', '8 to 15', '15 to 30', 'more than 30'],
    'Days_Policy_Claim': ['none', '8 to 15', '15 to 30', 'more than 30'],
    'PastNumberOfClaims': ['none', '1', '2 to 4', 'more than 4'],
    'AgeOfVehicle': ['new', '2 years', '3 years', '4 years', '5 years', '6 years', '7 years', 'more than 7'],
    'AgeOfPolicyHolder': ['16 to 17', '18 to 20', '21 to 25', '26 to 30', '31 to 35', '36 to 40', '41 to 50', '51 to 65',
    'NumberOfSuppliments': ['none', '1 to 2', '3 to 5', 'more than 5'],
    'AddressChange_Claim': ['under 6 months', '1 year', '2 to 3 years', '4 to 8 years', 'no change'],
    'NumberOfCars': ['1 vehicle', '2 vehicles', '3 to 4', '5 to 8', 'more than 8'],
}
for col, ordem in FAIXAS_ORDINAIS.items():
    df[col] = df[col].astype(str).str.strip().map({faixa: i for i, faixa in enumerate(ordem)})
# Age = 0 é um valor inválido na base (o motorista mais novo está na faixa 16 a 17 anos): tratado como ausente
df['Age'] = df['Age'].where(df['Age'] > 0)

# Categóricas -> One-Hot ('0' em DayOfWeekClaimed é valor desconhecido)
colunas_categoricas = ['DayOfWeek', 'DayOfWeekClaimed', 'Make', 'AccidentArea', 'Sex', 'MaritalStatus', 'Fault',
                       'VehicleCategory', 'PoliceReportFiled', 'WitnessPresent', 'AgentType', 'BasePolicy']
for col in colunas_categoricas:
```

```
    df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhecido').astype(str).replace('0', 'Desconhecido')
df = pd.get_dummies(df, columns=colunas_categoricas, drop_first=True, dtype=float)

# Numéricas ausentes (faixas fora da lista, Age = 0, mês desconhecido) -> mediana
for col in df.select_dtypes(include=np.number).columns:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].median())
print(f"Engenharia de features concluída: {df.shape[1] - 1} features.")
print("-" * 50)

print("\n--- 4. Preparando Dados para Machine Learning ---\n")

df_ml = df.copy()

# Separando X (Features) e y (Alvo)
X = df_ml.drop(columns=['FraudFound_P'])
y = df_ml['FraudFound_P']

# Divisão Correta: 60% Treino, 20% Validação, 20% Teste
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.25, random_state=42, stratify=y_temp)

print(f"Dados divididos! Treino: {X_train.shape[0]}, Validação: {X_val.shape[0]}, Teste: {X_test.shape[0]}")

# Escalonamento Robusto: Vital para o Isolation Forest não dar peso excessivo a valores anômalos
print("Aplicando RobustScaler para normalizar as grandezas numéricas...")
scaler = RobustScaler()
# O escalonador SÓ PODE ver os dados de treinamento
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

# Convertendo de volta para DataFrame por praticidade
X_train = pd.DataFrame(X_train_scaled, columns=X_train.columns)
X_val = pd.DataFrame(X_val_scaled, columns=X_val.columns)
X_test = pd.DataFrame(X_test_scaled, columns=X_test.columns)

print("\n--- 5. Simulação de Aprendizado Federado (FL) ---\n")
NUM_CLIENTES = 3
print(f"Simulando ambiente com {NUM_CLIENTES} clientes (silos de dados).")

# Divisão dos dados de treino entre os "clientes"
tamanho_chunk = len(X_train) // NUM_CLIENTES
clientes_X = [X_train.iloc[i * tamanho_chunk : (i + 1) * tamanho_chunk] for i in range(NUM_CLIENTES)]

modelos_locais = []
tempos_treinamento = []

print("\nIniciando Treinamento Local (Isolation Forest) nos clientes...")
for i in range(NUM_CLIENTES):
    inicio_treino = time.time()

    # IsolationForest OTIMIZADO:
    # Contaminação 'auto' permite que a floresta foque na estrutura intrínseca dos dados
    modelo_local = IsolationForest(
        n_estimators=150,
        max_samples=0.8,
        contamination='auto',
        n_jobs=-1,
        random_state=42+i
    )

    # Cliente treina apenas com sua fatia de dados
    modelo_local.fit(clientes_X[i])
    fim_treino = time.time()

    modelos_locais.append(modelo_local)
    tempos_treinamento.append(fim_treino - inicio_treino)
    print(f"-> Cliente {i+1} treinado em {fim_treino - inicio_treino:.2f} segundos.")

print("\n--- 6. Agregação Federada e Otimização de Limiar ---")

class ModeloFederado:
    def __init__(self, modelos_locais):
        self.modelos_locais = modelos_locais

    def decision_function(self, X):
        # Média dos escores de anomalia de todos os clientes locais
        escores = np.zeros(X.shape[0])
        for modelo in self.modelos_locais:
            escores += modelo.decision_function(X)
        return escores / len(self.modelos_locais)
```

```
modelo_global = ModeloFederado(modelos_locais)

# ESTRATÉGIA CORRIGIDA: Encontrar o melhor limiar no conjunto de VALIDAÇÃO
print("\nOtimizando o limiar de decisão via Curva PR no conjunto de VALIDAÇÃO (Sem Data Leakage)...")
# Invertemos o sinal: Valores MAIORES = maior probabilidade de fraude
y_scores_val = -modelo_global.decision_function(X_val)

# Calculando precisão e recall para vários limiares possíveis usando a Validação
precisions, recalls, thresholds = precision_recall_curve(y_val, y_scores_val)

# Calculando F1-Score para cada limiar (evitando divisão por zero)
f1_scores = np.divide(2 * (precisions * recalls), (precisions + recalls),
                      out=np.zeros_like(precisions), where=(precisions + recalls) != 0)

# O melhor limiar é aquele que maximiza o F1-Score na Validação
melhor_indice = np.argmax(f1_scores)
if melhor_indice < len(thresholds):
    limiar_otimizado = thresholds[melhor_indice]
else:
    # Prevenção contra vetor vazio
    limiar_otimizado = np.percentile(y_scores_val, 99)

print(f"Limiar matemático otimizado (F1 Máximo na Validação): {limiar_otimizado:.4f}")

print("\n--- 7. Inferência no Servidor Global (Teste Real) ---")
inicio_inferencia = time.time()

# Aplicando os escores estritamente na base de TESTE
y_scores_test = -modelo_global.decision_function(X_test)

# Classificando baseado no limiar otimizado, sem que o modelo tenha visto os dados
y_pred = (y_scores_test >= limiar_otimizado).astype(int)

fim_inferencia = time.time()
tempo_inferencia = fim_inferencia - inicio_inferencia

print("\n--- 8. Resultados Globais Pós-Otimização ---\n")

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, zero_division=0)
rec = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
roc_auc = roc_auc_score(y_test, y_scores_test)
pr_auc = average_precision_score(y_test, y_scores_test)

print(">> MÉTRICAS DE QUALIDADE DO MODELO (ISOLATION FOREST):")
print(f"Acurácia Geral      : {acc * 100:.4f}%")
print(f"Precisão (Precision): {prec * 100:.4f}%")
print(f"Revocação (Recall)  : {rec * 100:.4f}%")
print(f"F1-Score            : {f1 * 100:.4f}%")
print(f"ROC-AUC             : {roc_auc:.4f}")
print(f"PR-AUC              : {pr_auc:.4f}")

tps = len(X_test) / tempo_inferencia if tempo_inferencia > 0 else float('inf')
print("\n>> MÉTRICAS DE RENDIMENTO TÉCNICO (THROUGHPUT):")
print(f"Amostras Processadas   : {len(X_test)} transações")
print(f"Tempo Total de Infe.   : {tempo_inferencia:.4f} segundos")
print(f"Rendimento (Throughput): {tps:.2f} transações por segundo\n")

# Visualizações
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 1. Matriz de Confusão (Aplicada sobre os dados de Teste)
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=['Normal', 'Fraude'], yticklabels=['Normal', 'Fraude'])
axes[0].set_title('Matriz de Confusão (Teste Final)')
axes[0].set_ylabel('Real')
axes[0].set_xlabel('Previsão')

# 2. Curva Precision-Recall (Do conjunto de Validação, onde o limiar foi escolhido)
axes[1].plot(recalls, precisions, color='purple', lw=2, label=f'PR-AUC (Val) = {average_precision_score(y_val, y_scores_v
# Proteção caso arrays estejam vazios ou melhor_indice fora dos limites
if len(recalls) > 0 and len(precisions) > 0 and melhor_indice < len(recalls):
    axes[1].scatter(recalls[melhor_indice], precisions[melhor_indice], color='red', s=100, label='Limiar Escolhido')
axes[1].set_xlabel('Recall (Revocação)')
axes[1].set_ylabel('Precision (Precisão)')
axes[1].set_title('Curva Precision-Recall (Dados de Validação)')
axes[1].legend(loc="lower left")

plt.tight_layout()
```